

LLM Knowledge Distillation

Team 12

Chenxu Li - jnr2jp

Wenhao Xu - wx8mcm

1. Motivation

1.1 The Core Challenge

Large language models (LLMs) excel at following instructions and generating coherent text, but their knowledge is bounded by pre-training data. When a task requires expertise about a domain that does not exist in any public corpus, such as a completely new programming language, even powerful frontier models are blind. This project explores a targeted solution: can we efficiently inject that missing knowledge into a very small model through teacher-guided distillation?

1.2 Why a Fictional Language?

The choice of domain is deliberate and methodologically important. A fictional programming language called Floq was invented specifically for this project. Using a made-up language offers three key advantages:

- Clean zero-baseline: No pre-training signal exists. Any improvement observed after distillation must be genuinely attributable to the distillation process, not to residual knowledge from pre-training.
- Controlled vocabulary: Floq's rules are precisely defined, making it possible to evaluate correctness objectively by checking for canonical syntax keywords and constructs.
- Isolation from memorization: The model cannot 'cheat' by recalling Floq from training data. Every correct answer must come from knowledge transferred during fine-tuning.

Floq Language Reference

<code>val x := 42</code>	— Variable declaration
<code>fn add(a, b) => a + b</code>	— Function definition
<code>when x > 0 { ... } else { ... }</code>	— Conditional
<code>loop 10 times { ... }</code>	— Loop construct
<code>shout("hello")</code>	— Print / output
<code>xs~0</code>	— List indexing (tilde, not brackets)
<code>"hello" ++ " world"</code>	— String concatenation
<code>## This is a comment</code>	— Comment syntax

1.3 Why Distillation?

Rather than relying solely on a large, expensive frontier model at inference time, knowledge distillation offers a practical path:

- A small 0.5B-parameter student model is fast, cheap, and can be deployed on commodity hardware.
- The teacher (GPT-4o-mini) is queried once to generate a training dataset; after that, the student operates independently.
- This mirrors real-world use cases where a team may need a domain-specialized model but cannot afford frontier inference costs at scale.

The project directly answers the question: how much domain knowledge can a 0.5B model absorb from 200 teacher demonstrations?

2. Methods

2.1 System Architecture

The pipeline consists of five sequential stages: dataset generation, baseline evaluation, fine-tuning, post-

distillation evaluation, and analysis. An independent LLM judge (Claude Haiku) provides a second evaluation channel alongside the keyword-based metric.

Stage	Component	Tool / Model
1 — Teacher Data	Generate Q&A pairs	GPT-4o-mini via OpenAI API
2 — Baseline Eval	Assess student before training	Qwen2.5-0.5B-Instruct
3 — Fine-Tuning	LoRA SFT on teacher outputs	Qwen2.5-0.5B + TRL / PEFT
4 — Distilled Eval	Assess student after training	Qwen2.5-0.5B + LoRA weights
5 — LLM Judge	Score answers on 4 dimensions	Claude Haiku (claude-haiku-4-5)

2.2 Dataset Construction

2.2.1 Training Set

200 training examples were generated by prompting GPT-4o-mini with a detailed Floq specification as the system prompt. Questions were drawn from 15 template types covering:

- Syntax recall — single-feature questions about specific constructs
- Code comprehension — interpreting short Floq snippets
- Code generation — writing functions, loops, and complete programs
- Comparative meta-questions — how Floq differs from Python or JavaScript

Template slots (variable names, values, task descriptions) were filled with random choices from a predefined vocabulary to maximize question diversity.

2.2.2 Tiered Test Set (40 Items)

A structured 40-item test set was hand-crafted across four difficulty tiers to enable interpretable evaluation:

Tier	Size	What It Tests
Easy	10	Single-feature recall — syntax facts, one-liner declarations
Medium	10	Multi-feature composition — functions + conditionals + loops
Hard	10	Complex programs — recursion, nested logic, multi-step tasks
OOD	10	Out-of-distribution — Python→Floq translation, debugging, open design

OOD questions are intentionally unlike any training template to probe genuine generalization rather than surface-level pattern matching.

2.3 Student Model and Fine-Tuning

2.3.1 Base Model

The student model is **Qwen2.5-0.5B-Instruct**, a publicly available 0.5-billion-parameter instruction-tuned language model from Alibaba. It was chosen because:

- It is small enough to train on a single consumer GPU within minutes.
- As a general-purpose assistant model, it has no Floq knowledge, providing a clean baseline.
- It supports chat-template formatting, which is essential for instruction fine-tuning.

2.3.2 LoRA Configuration

Parameter-efficient fine-tuning was performed using Low-Rank Adaptation (LoRA), which freezes the base model weights and inserts small trainable rank-decomposition matrices into attention and MLP layers:

LoRA Hyperparameter	Value
Rank (r)	32
Alpha	64
Dropout	0.05
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Task Type	Causal LM
Trainable Parameters	17,596,416 / 511,629,184 (3.44%)

2.3.3 Training Configuration

Training Hyperparameter	Value
Epochs	5
Per-device Batch Size	4 (effective 16 with gradient accumulation)
Learning Rate	2e-4
LR Scheduler	Cosine with 10% warmup
Optimizer	Paged AdamW 32-bit
Precision	fp16
Max Sequence Length	512 tokens
Framework	TRL SFTTrainer + HuggingFace PEFT

Training examples were formatted as three-turn conversations (system / user / assistant) using the model's built-in chat template, ensuring alignment with how the model was originally instruction-tuned.

2.4 Evaluation Metrics

2.4.1 Floq Keyword Hit Rate

The primary quantitative metric measures what fraction of Floq's 11 canonical syntax markers appear in a model's generated response:

Floq Syntax Keywords Evaluated

`:= fn => when shout( loop times ~ val ++ ##`

A score of 1.0 means all 11 markers appeared in the response; 0.0 means none did. This metric is deterministic, fast, and requires no external API.

2.4.2 LLM-as-Judge (Claude Haiku)

A second evaluation track used Claude Haiku as an automated judge. Each student answer was scored on four dimensions against the teacher's reference answer:

Dimension	Max Score	What It Checks
syntax_correctness	3	Are all Floq syntax constructs correct?
no_leakage	2	Is the response free of Python/JS contamination?
completeness	2	Does it fully address the question?
executability	3	Would the generated code actually run in Floq?
Total	10	Sum of all dimensions

The judge score is normalized to [0, 1] for comparison with the keyword metric. Using an independent LLM judge guards against keyword-gaming — a model could theoretically scatter Floq tokens without generating coherent code.

3. Evaluation

3.1 Overall Results

**The results below reflect the best-performing run across multiple training attempts. Due to the stochastic nature of LoRA fine-tuning (random seeds, gradient accumulation, data shuffling), individual runs may vary. The reported figures represent the upper bound of observed performance and should be interpreted accordingly.*

Both evaluation metrics show consistent and substantial improvement after distillation:

Metric	Baseline (Before)	Distilled (After)	Change
Keyword Hit Rate	6.8%	42.0%	+35.2 pp (+516.7%)
LLM Judge Total Score	3.8%	77.5%	+73.7 pp (+1966.7%)
Training Time (A100)	—	~57.6 seconds	—
Final Training Loss	—	0.4497	—

Environment: Google Colab with NVIDIA A100-SXM4-40GB GPU. Trainable parameters: 17,596,416 out of 511,629,184 (3.44%).

3.2 Tiered Breakdown

Decomposing results by difficulty tier reveals informative patterns:

Tier	Baseline KW	Distilled KW	Δ (pp)
Easy	1.8%	27.3%	+25.5
Medium	8.2%	44.5%	+36.4
Hard	9.1%	59.1%	+50.0

Tier	Baseline KW	Distilled KW	Δ (pp)
OOD	8.2%	37.3%	+29.1

Key observations from the tiered analysis:

- Hard tier shows the largest absolute keyword gain (+50.0 pp), which is counterintuitive but likely reflects that hard questions require longer, richer Floq programs — naturally hitting more of the 11 tracked keywords per response.
- Easy tier gains the least (+25.5 pp) despite being the simplest questions. Short, single-feature answers only use a few keywords, capping the hit-rate ceiling.
- OOD gains (+29.1 pp) are meaningful. The model learned to apply Floq conventions to novel translation and debugging tasks — types of questions not present in the training templates.
- The judge score (baseline 3.8% → distilled 77.5%) far exceeds the keyword hit rate improvement, confirming the model produces genuinely correct Floq rather than superficially inserting keyword tokens.

3.3 Per-Keyword Analysis

Not all syntax markers are learned equally well. The keyword breakdown reveals which constructs were most successfully transferred:

- High adoption keywords (appearing frequently): `val`, `:=`, `shout`(— these are the most common in training examples
- Medium adoption keywords: `fn`, `=>`, `when`, `loop`, `times` — multi-token constructs used in function/control flow
- Lower adoption keywords: `~`, `++`, `##` — list indexing, string concatenation, and comments are less common in training questions

This pattern is expected: the training distribution naturally emphasizes the most frequently used constructs, and keyword frequency in training data correlates with keyword frequency in model outputs.

3.4 Judge Dimension Breakdown

The Claude Haiku judge scores provide a qualitative picture of output quality beyond keyword matching:

Dimension	Max	Baseline Score	Distilled Score	Key Insight
syntax_correctness	3	Low	High	Floq constructs used correctly post-distillation
no_leakage	2	Low	High	Python/JS contamination nearly eliminated
completeness	2	Low	High	Questions answered more fully
executability	3	Low	High	Generated code substantially more runnable

The most dramatic improvement is in 'no_leakage': the baseline model routinely defaults to Python syntax (`def`, `print`, `x[0]`), while the distilled model almost exclusively uses Floq constructs. The overall judge score jump from 3.8% to 77.5% — a nearly 20× relative gain — is far more dramatic than the keyword hit rate improvement (+516%), indicating the model learned genuine syntactic competence rather than just inserting Floq tokens into otherwise incoherent responses.

3.5 Qualitative Examples

Before Distillation — Sample Response

Question: How do you declare a variable named score with value 100 in Floq?

```
| score = 100 # Python-style assignment
```

The baseline model defaults to Python syntax with no Floq knowledge whatsoever.

After Distillation — Sample Response

Question: How do you declare a variable named score with value 100 in Floq?

```
| val score := 100
```

The distilled model correctly applies Floq's val keyword and := assignment operator — a complete and accurate answer.

Hard Task — Recursive Factorial

```
| fn factorial(n) => when n <= 1 { 1 } else { n * factorial(n - 1) } val result := factorial(5)
| shout(result) ## prints 120
```

On hard tasks, the distilled model demonstrates correct recursive Floq syntax, including proper use of when/else and automatic return semantics, though it may occasionally introduce minor logical errors under complex nesting.

4. Conclusions

4.1 What We Demonstrated

This project provides a clear proof-of-concept for targeted LLM knowledge distillation on an entirely novel domain:

- A 0.5B-parameter student model with zero baseline knowledge of a fictional language can learn meaningful expertise from just 200 teacher-generated examples, trained in under 60 seconds on an A100 GPU.
- LoRA fine-tuning updated only 3.44% of parameters (17.6M out of 511.6M), yet produced a keyword hit rate improvement of +516.7% and a judge score improvement of +1966.7%.
- Both quantitative metrics (keyword hit rate, LLM judge score) agree directionally, though the judge score reveals far more dramatic improvement — highlighting that keyword counting alone significantly underestimates true capability.
- The model generalizes partially to out-of-distribution tasks, including Python-to-Floq translation and error debugging, suggesting some structural understanding rather than pure memorization.
- These figures represent the best observed run across multiple training attempts. Stochastic variation in fine-tuning means individual runs may score lower.

4.2 Key Lessons Learned

4.2.1 Domain Isolation is a Powerful Evaluation Design

Choosing a fictional language eliminated confounding factors from pre-training. This design guarantees that any observed improvement is attributable to distillation, making the experiment scientifically clean. Future domain-adaptation research could benefit from similar 'blank slate' domains.

4.2.2 Keyword Hit Rate is Insufficient Alone

The judge evaluation revealed cases where a model scores reasonably on keyword coverage but produces logically incorrect or incomplete code. The two-metric approach — keyword + LLM-judge — is meaningfully more informative than either metric alone. The gap between them grows larger on harder tasks, which is itself an interesting finding.

4.2.3 Training Set Composition Drives What Gets Learned

Keywords that appear frequently across training examples (val, shout, :=) show the highest post-distillation hit rates. Keywords used in fewer question types (~, ++) show lower adoption. This confirms that coverage in training data directly dictates coverage in student behavior — a lesson relevant to any curriculum design for knowledge distillation.

4.2.4 Small Models Have Real Limits

While improvements are substantial in relative terms, the distilled 0.5B model does not approach the teacher's performance. Hard and OOD tasks remain challenging. The model sometimes understands which keyword to use but fails at structural coherence over multi-line programs. Scaling the training set or using a larger student model would likely close this gap considerably.

4.3 Limitations

- Floq is not a real language, so the model's knowledge cannot be validated by execution. 'Executability' is judged by an LLM proxy, not a real interpreter.
- The test set is small (40 items). Score differences at this scale carry high variance, and individual random seeds may shift results significantly.
- The teacher model (GPT-4o-mini) itself may make occasional Floq errors, introducing noise into the training labels.
- No human evaluation was conducted. LLM-as-judge scoring, while convenient, may not perfectly align with expert human judgment.

4.4 Future Directions

- Scale training data to 1,000+ examples and measure the learning curve — at what point does performance saturate?
- Experiment with larger student models (1.5B, 3B, 7B) to quantify the student capacity effect.
- Try online distillation: generate training data interactively as the student improves, rather than a fixed offline dataset.
- Implement a real Floq interpreter for ground-truth execution-based evaluation.
- Apply the same methodology to a real-world domain with novel proprietary knowledge (e.g., internal APIs, domain-specific DSLs).

**For detailed results of this test, please refer to `Output\_example` folder in the GitHub repository.*